

System 14: In-Memory Pub/Sub Message Broker (Kafka Lite)

Zero-Prerequisite Intuition: The "Newspaper Subscription" Metaphor What is a Publish/Subscribe (Pub/Sub) message broker, and why did the software industry invent it? Imagine a news agency that prints financial news. If the news agency had to personally call 50,000 individual readers on the telephone every time a story broke, the journalists would spend 100% of their day dialing phones. If 5,000 new readers sign up, the journalists are completely overwhelmed.

Instead, the agency prints newspapers and drops them into a central **Newsstand (The Topic)**.

- **Publishers (Journalists):** Drop articles into the newsstand and immediately walk away.
- **Subscribers (Readers):** Walk up to the newsstand whenever they are free, read the articles at their own speed, and remember which page they finished reading (**The Offset**).

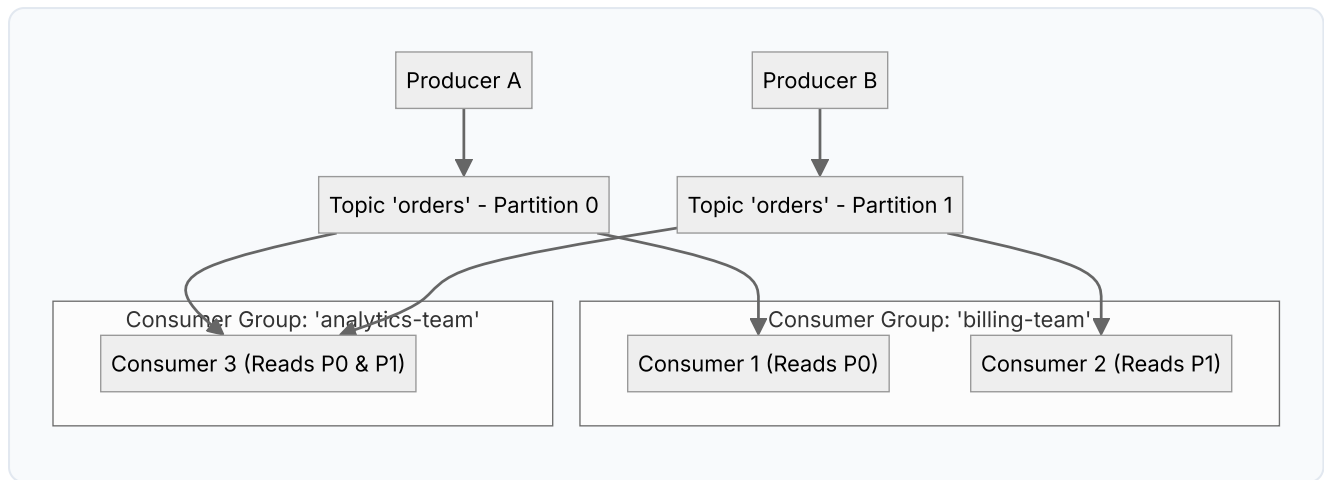
In microservices (Amazon, Netflix, Uber), Order Services don't call Billing, Shipping, Analytics, and Fraud directly. They publish an `OrderPlaced` event to a message broker, and each downstream team consumes it independently!

This chapter designs an **In-Memory Pub/Sub Message Broker** (a simplified Apache Kafka engine) in pure Python.

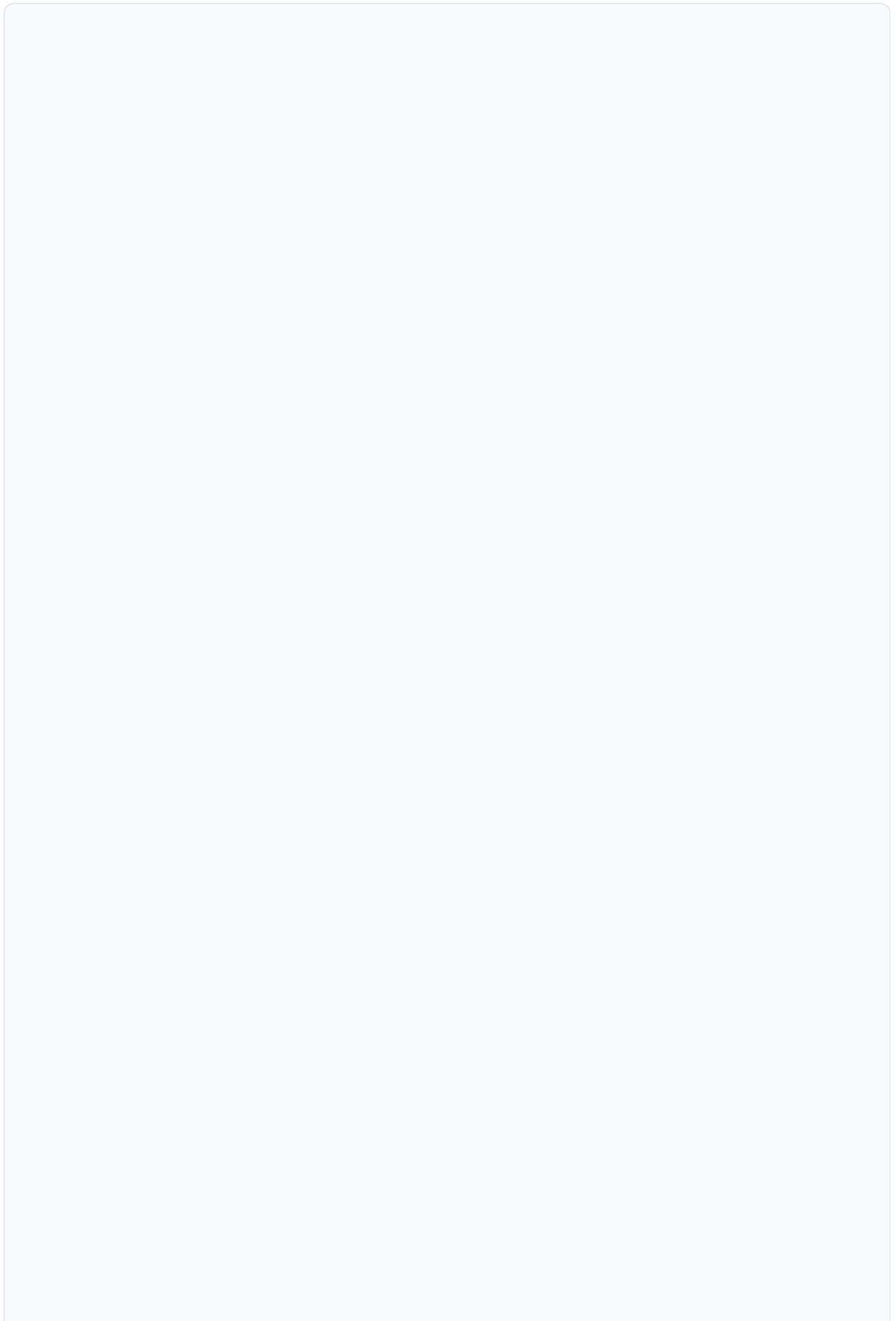
1. Requirements & Core Concepts

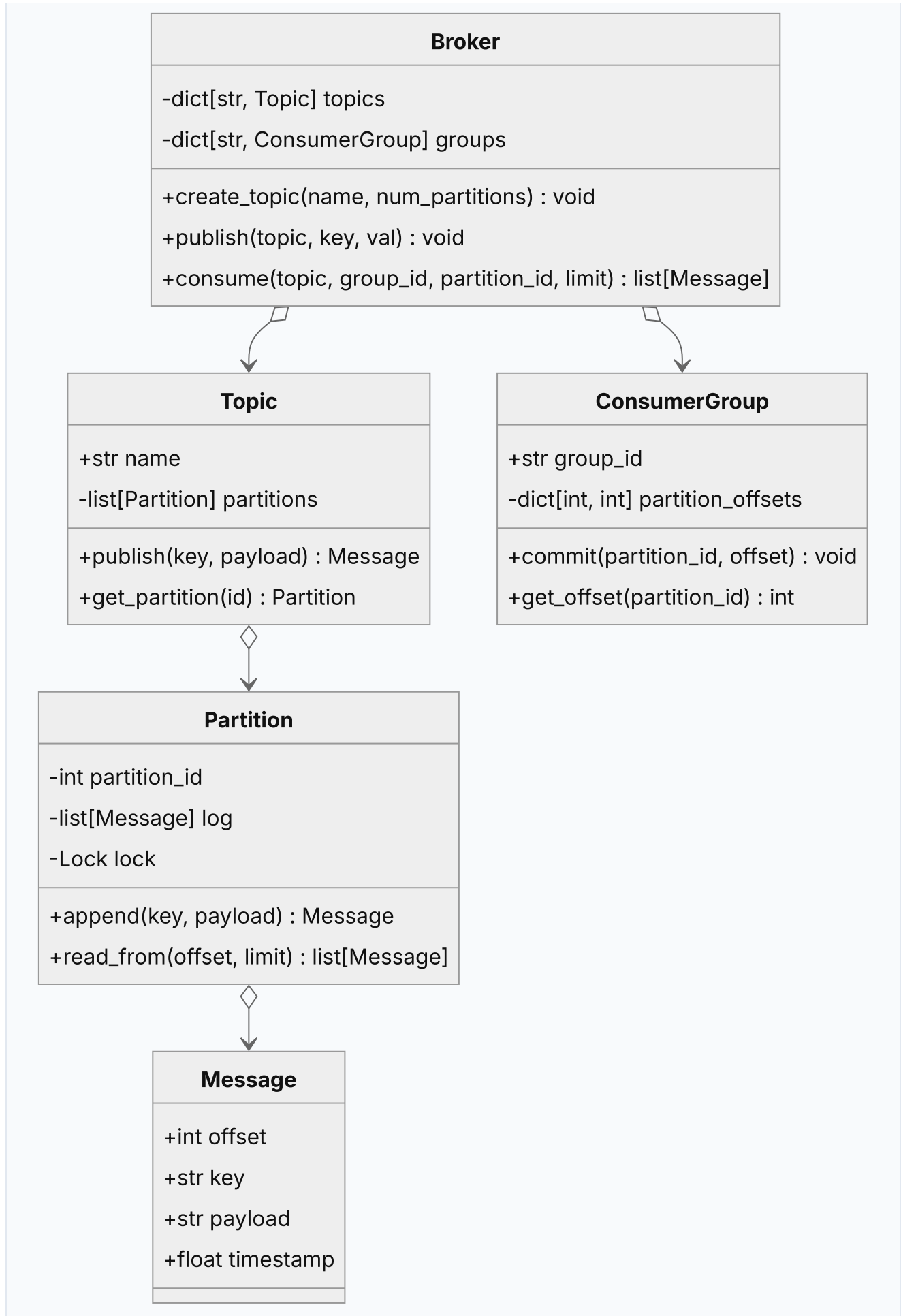
Core Primitives Defined

1. **Message:** An immutable payload with a `key`, `value`, `timestamp`, and assigned `offset`.
2. **Topic:** A named logical stream of messages (e.g., `user-signups`).
3. **Partition:** A strictly ordered, append-only log within a topic that guarantees message order.
4. **Consumer Group:** A collection of consumers cooperating to read data. Each message sent to a topic is delivered to **one consumer instance within each subscribing consumer group**.



2. Design Pattern & Class Hierarchy





3. Complete Python Implementation



4. Verification: Multi-Consumer Group Isolation



Both consumer groups read the exact same data without interfering with each other's reading progress!

4. Edge Cases, Tests and Extensions

Semantics and the weak spots

Concern	Status	Detail
Per-key ordering	Good	Same key goes to the same partition, and a partition is an append-only log
Independent consumer groups	Good	Each group has its own offsets, so fraud and analytics both see every message
Key to partition mapping	Bug across restarts	<code>hash(key)</code> for a <code>str</code> is randomised per process (hash randomisation), so after a restart the same key can land on a different partition and order is lost
Keyless messages	Hot partition	<code>key=None</code> always goes to partition 0; round-robin spreads the load
Offset committed on read	At-most-once	<code>consume</code> advances the offset before the caller has processed the batch; a crash loses messages. Kafka separates <code>poll</code> from <code>commit</code>
Two consumers in one group on one partition	Duplicates	<code>get_offset</code> and <code>commit_offset</code> are separate lock acquisitions, so both can read the same batch
Retention	None	The log grows forever; real brokers delete or compact by size, time or key

Tests

This block extends the implementation above. It starts subprocesses with different `PYTHONHASHSEED` values to show the partitioning bug for real.

Extensions interviewers ask for

1. **At-least-once with explicit commit:** split `consume` into `poll` (returns messages and a token) and `commit(token)`; on restart the group resumes from the last commit, so messages may repeat and handlers must be idempotent.
2. **Consumer group membership:** assign partitions to the consumers of a group with the rebalance logic from the Kafka case study, so two consumers never read the same partition.
3. **Retention and compaction:** delete segments older than N hours, or keep only the latest message per key for a "current state" topic.

4. **Backpressure and slow consumers:** because consumers pull, a slow one just lags; expose lag (`log_end_offset - committed_offset`) as the key health metric.
5. **Delivery guarantees summary:** at-most-once (commit before processing), at-least-once (commit after), effectively-once (at-least-once plus idempotent processing or transactions).

Follow-up questions

- *Why partition at all?* One log has one writer-ordering point; partitions give parallelism while preserving order per key, and are the unit of consumer scaling.
- *What limits consumer parallelism?* The partition count: a group cannot usefully have more active consumers than partitions.
- *Why must the key hash be stable?* Ordering per key depends on the key always reaching the same partition; a hash that changes between runs silently breaks it.